National Research University Higher School of Economics

Faculty of Computer Science
Programme: Data Science and Business Analytics

BACHELOR'S THESIS
Software Project
Development of a System for a Short-term Forecasting of Time Series

Prepared by the student of Group 213, Year 4 (year of study),
Sinitsin Ivan Maksimovich

Thesis Supervisor:
School of Data Analysis and Artificial Intelligence, Senior Lecturer,
Parinov Andrey Andreevich

Moscow
2025

Table of Contents

Abstract 4

List of Key Words 4

Basic terms and definitions 5

1 Introduction 6

1.1 Relevance 6

1.2 Project Purpose 7

1.3 Project Goals 8

1.3 Summary of the work 9

2 Review of alternatives on the market 10

2.1 KNIME 10

2.2 Microsoft Power BI 11

2.3 Google Colab 11

2.4 Streamlit 12

2.5 Gradio 13

2.6 DataRobot AI Platform 13

2.7 PYTSBE 13

2.8 Conclusions and Results 15

3. Requirements 16

3.1 Functional Requirements 16

3.2 Non-Functional Requirements 17

3.3 Conclusions and Results 17

4 System Architecture 17

4.1 Technological stack 17

4.2 Other Components Considered 19
4.2.1 Plain Django 19
4.2.2 SvelteKit 20
4.2.3 Flask 20
4.2.4 NoSQL Databases (MongoDB, Apache Cassandra, …) 20
4.2.5 InfluxDB 21
4.2.6 Apache Pinot 22
4.2.7 TimescaleDB 22
4.3 Constructional Patterns 23
4.3.1 Site Layout and Feature Overview 23
4.3.2 Security 27
4.3.3 Data Processing Pipeline Editor 29
4.3.4 User Interface Design 30
4.4 Data Processing Architecture Overview 31
4.5 Conclusions and Results 36
5. Models 36
5.1 Models Utilized 36
5.2 Performance Comparison 37
5.3 Conclusions and Results 37
6. Conclusion 38
6.1 Further Development 39
7. List of Sources 39

Abstract

In the modern day, with steady increase in demand for the matter of analytical work to be conducted in various industries, the need for more accessible tooling becomes more evident. As more tools arise to meet this demand, the portion related to general time series workloads appears to remain lacking.

The project presents itself as a software product, aiming towards performing tasks of short-term forecasting of time series data. The system developed is capable of working with various types of models, operating off of user supplied datasets supplied through the functionality of the system, allowing for its users to construct data processing and forecasting pipelines.

The users are able to perform their work on multiple projects, all managed within the system itself, allowing for sharing of works with access control. The established user interface is designed for proper operation on mobile and desktop devices alike, ensuring its cross-platform availability, as well as its ease of use. The system requires no programming skills from the users for the purposes of analytical work, and is built with expandability in mind, ensuring that its feature set can be further extended to suit additional workloads, as it is built with Python programming language, enabling integration with its library ecosystem.

The development of the system has been accomplished through the utilization of Django framework on the backend, with PostgreSQL as its primary database, as well as a mix of CSS, HTML, JavaScript and Svelte composing the frontend part of the project.

List of Key Words

Time Series Forecasting, Time Series Analysis, Analytical Platform, Web application, Full-Stack Project Development

Basic terms and definitions

AI (Artificial Intelligence) – generally referred to the generative machine learning models, it is a collection of tools and services offering production of text and media information upon user's demand.

API (Application Programming Interface) – a connecting interface between different systems realized in a form of a set of functionalities in a particular application, allowing for it to communicate with other applications of interest over a particular protocol (in case of a Web Application, this typically encapsulates communication of the Frontend and the Backend, as well as the Backend with Databases).

Backend – part of a Web Application (or otherwise) ensuring the execution of the code by the server. Generally, it is responsible for the communication with the Database, as well as additional functionality that is in need to be in a centralized place.

Component – part of an application, the architecture of which adheres to the Component-Based Development style, in which it represents a reusable part of the code, coupled with similar parts of the code, utilized for the construction of the general application. Each component is responsible for either rendering specific parts of the application, taking care of self-contained logic, or otherwise.

Frontend – part of a Web Application ensuring the execution of the code by the client (typically that being user's web browser). It is responsible for handling logic on the user's device.

Library – an additional piece of software / programming code utilized for aiding in the development process of the final product.

Technological Stack – a common set of tools utilized in the construction and further development of a particular software project. Generally, it

encapsulates large frameworks & libraries that describe the nature of the application's behavioral nature and similar.

SPA (Single Page Application) – a Web Application realized in a form of a single page (effectively remaining on a single URI at all times).

UI (User Interface) – graphical representation of the application with which the user interacts.

Web App (Web Application) – software that is accessible over the web via web browsers, usually deployed in a regular form or an SPA.

1 Introduction

1.1 Relevance

In the modern day & age, the demand for accessible data science workflows arises, as the analytical field gains more and more traction with the advent of new AI technologies, as well as the growth of analytical work needed to be done in various professional fields. From this comes the need for more analytical platforms, as more and more people get into the field of data science, as well as the demand for simpler and easier workflows whenever that can be achieved, in order to simplify the work being conducted by the analysts. Throughout the recent years quite a few solutions have emerged attempting to address this issue, simplifying general aspects of managing data processing and analysis, yet most of these tools require either significant investment of time, effort and skill to properly operate. As well as that, the general focus of such systems being brought forth to the market generally varies quite a lot, each focusing on very specific aspects of the analytical work (visualization, data processing, machine learning, etc), and as such, certain fields like time series analysis get left out.

With this, there also comes the issues of accessibility and ease of use, as well as complications on the matter of feature expandability and system adaptability to one's particular needs. As such, this project is called on forth to attempt to remedy such issues in certain part by filling in a niche of performing kinds of analytical and forecasting work, as well as conducting model assessment and more with relative ease on the part of user experience.

1.2 Project Purpose

The primary goal of the project is the development of an online platform which enables construction of data processing pipelines, allowing to perform various kinds of workflows as deemed necessary, utilizing the models and processing implemented, with primary focus set on supporting predictive workflows with time series, and with a specific focus upon allowing to accomplish such goals without the need for the users to write code on their side. The constructive work is conducted with the tools provided to the users in a form of a web application, all accomplished with a graphical user interface, available across different platforms.

This web application is constructed in a form of a system, in which users are able to manage their currently worked on projects via general storage of metadata information, as well as share it with other users present within the aforementioned system through making their projects public to browse and observe. Additional features for profile management, search and similar are brought forth to aid users in this task.

Lastly, this project is built with expandability of workload complexity in mind, bringing forth the underlying structure allowing for the general ease of feature set expansion via the utilization of conventional programming environments and underlying structure to achieve said results.

1.3 Project Goals

- Review existing solutions available on the market, which offer similar functionality to this project's objective, compare and contrast their features, derive their strong and weak sides.
- Construct a general mechanism by which the application operates, utilizing modern tooling to achieve said goal.
- Develop application's backend structure that is capable of supporting the workflows required.
- Establish proper way of handling and accessing data within the system.
- Develop a system for enabling the runtime of data processing functions on the backend, ensuring the availability of Python's data processing ecosystem [2], as well as taking care of the functionality's runtime stability and expendability.
- Design a modern looking UI for users to interact with throughout the application.
- Construct an SPA on the frontend side of the system, which presents the editing functionality, allowing for the construction of data processing pipelines, as well as forecasting of time series to the users via a web browser, while also adheres to the design language of the application.
- Build an API taking care of communication between the aforementioned SPA and the Backend for the purposes of proper interoperability of both parts of the system.
- Extend the feature set of the service through development of various data processing functions, including the models capable of performing the workloads on the matter of the forecasting of time series, for the purposes of demonstrating capabilities of the developed system. Such functions and models must remain

compliant with the underlying system architecture, in order to ensure their proper usability on the side of the users.

1.3 Summary of the work

By the end of the project's development, a platform designed for the purposes of performing predictive time series workloads, as well as analysis and processing of time series data is achieved. It supplies the tools for editing data processing pipelines, dataset management, profile management and handling the projects within the system with storage of additional metadata helpful for saving specific details about the work, as well as controlling their visibility and access parameters. It also serves as a template for further expansion of the functionality based on one's particular needs, making sure to present as little technical limitations on the matter of adding new functions as possible.

The system is developed and designed in a form of a website, accessible from different kinds of devices, with its interface properly accommodating for both desktop and mobile users. Within the system they are able to conduct analytical work without the need to write any code on their part.

The design of the system, its features and goals have been constructed based upon the conducted in chapter 2 "Review of Alternatives on The Market", in which strengths and weaknesses of existing solutions are assessed for the purposes of this project to attempt to remedy the shortcomings of other existing works. After conducting this analysis, the Requirements for the project are established. Continuing on, the system architecture is described in chapter 4 "System Architecture", bringing forth the detailed overview of existing tooling available for achieving set out goals, creating proper constructional patterns, and implementing all of the described work in a form of a Web Application. Lastly, the model work is being added into the system for the purposes of demonstrating the system's capabilities on the matter of performing time series analysis workflows, including model assessment and their forecasting functions.

2 Review of alternatives on the market

Under the development of this project, a number of existing software solutions, aiming to provide similar services, has been reviewed. As well as that, with the composition of the technological stack, various solutions on offer have been assessed as well.

2.1 KNIME

KNIME (Konstanz Information Miner) [5] is an open-source tool designed for the purposes of data analytics work, reporting and integration, which offers visual construction of data processing pipelines, model training and more. The toolset that it provides allows for the creation of various data processing pipelines, as well as extensive data visualization and simple model training, all accomplished via a graphical interface accessed by its users.

Its downside, however, is that it is only available on desktop operating systems, excluding mobile platforms, which is problematic due to the rising prominence of the latter in the modern day. Furthermore, it is a fully local solution, which makes it bound by the end user's device capabilities in terms of performance. Lastly, the fact that it is written in Java comes as an obstacle on the matter of the expandability of its feature set with the development of additional modules for the application, as most of the data science workloads are being handled through utilization of other programming languages like Python, widely utilized in the data science field. This comes as a problem, particularly for the matter of time series analysis and forecasting work, since it doesn't feature any of such models for such purposes within itself, with only a few modules available on the marketplace supplied by the community, which this project's goal is to remedy with much easier feature set expansion with support for existing Python solutions available.

2.2 Microsoft Power BI

Microsoft Power BI (Microsoft Power Business Intelligence) [9] is a software kit designed for the purposes of processing, analysis and visualization of various kinds of data, with the primary focus set upon the management of Business Information, offering an extensively integrating solution into company systems for the purposes of constructing insights on business's operational data and similar, as well as automation of various kinds of tasks within the aforementioned systems. Within its feature set, aside from the described tooling, it also offers various visualization workloads, helpful for certain kinds of analytical workloads, and more.

Whenever things come to predictive analysis, however, this solution falls flat, as its primary focus lies within proper digestion and visualization of existing data, as well as composing reports for further analysis, whereas more extensive model-based workloads remain largely excluded from its general workloads. For the purposes of time series analysis in particular, it does a fine enough job with construction of graphs and charts, but not much else. Furthermore, it's a proprietary product geared towards use with Microsoft's own product ecosystem, which complicates the matter of adaptability to specific systems, as well as binding its use cases by the imposed restrictions from the corporation, largely requiring additional payments for more extensive use of the solution. The latter is especially problematic nowadays, in large due to the increased complexity of the product's availability and acquisition process for entities based within the Russian Federation.

2.3 Google Colab

Google Colab (full name – Google Colaboratory) [6] is a powerful development environment utilized by many Python and R programmers to accomplish the purposes of data science studying and development for various purposes. The platform by itself presents as a cloud-based wrapper around the already popular Jupyter Notebook, which creates a programming pattern of a

shell-like workflow with additional features allowing for readable displays of data, graphs and other things the work upon which may concern.

The major downside of this platform is that, unlike the others that are being reviewed, this particular platform's focus is purely on experienced developers and data scientists well knowledgeable to do proper programming, which greatly increases the issue of accessibility and availability of the tool for people not familiar with the required programming languages, libraries or otherwise. It serves as more of a regular programming environment instead, complicating entry to the data science space. It is worth noting, however, that since the system that is being developed is implemented in Python, it is therefore capable of taking advantage of the same code that is being developed in the form of Jupyter Notebooks and is being hosted on platforms like Colab, therefore it can be integrated as separate functions and modules into the application with relative ease.

2.4 Streamlit

Streamlit [7] is a general Python-based web framework allowing for building of applications generally dedicated towards data processing, display and similar. It aims to be an approachable solution for the model displays and similar workflows centered around various things related to data science and is popular for demos and similar tasks.

The notable downside of Streamlit is the fact that it is almost fully a self-contained tool, which does not integrate with other workflows and is only really useful for data visualization and model demonstration. Furthermore, while it strives towards maximal simplification and embracing of simple scripting language for writing the application logic, similar to Colab it remains to be a highly code-dependent platform, which remains an issue with certain workflows.

2.5 Gradio

Gradio [8] is a minimalistic web framework similar to that of the already covered Streamlit, providing easy tooling for the construction of web UIs for interacting with machine learning models. Initially considered for the purposes of graphing workloads within the application, it was determined that, while it's not as self-contained as Streamlit, it still lacks in terms of its interoperability with external systems, requiring additional runtime to be executed in parallel to the main application, limiting its interactivity and hindering performance of both systems. Furthermore, its focus on machine learning and AI workloads makes it even less applicable in this particular project.

2.6 DataRobot AI Platform

The DataRobot AI Platform [10] is a cloud-based AI development and integration platform, which allows for construction, testing and deployment of different kinds of models in a business setting, allowing for their integration into the business process. As part of their functionality, they offer a multitude of analytical tools and models, including time series workloads (although the primary goals of such a platform remains to be machine learning workloads, therefore the models present are based on this concept).

Unfortunately, such a service is fully proprietary and not free to use, only having a fairly short trial on offer for the matter of 14 days. Going further, it is a, once again, a fully self-contained system, employing its own system of model construction, which restricts additional feature set expansion with tooling provided in various kinds of Python frameworks and libraries available.

2.7 PYTSBE

As one of the use cases of a forecasting time series system can support, one particular task that might be of interest towards its users is the general model comparison and assessment of performance. One of such systems geared towards performing this particular task is the Python time series benchmark

project [21], developed and brought fourth for the general matter of assessing various models from different Python libraries.

As worthy of note, however, is that such a system is only geared towards the matter of performing various benchmarks, instead of focusing on actual predictive workloads, and is as such falling short of meeting the requirements for performing forecasts and similar workloads. Furthermore, it's not particularly user friendly, requiring direct interaction with code.

2.8 Conclusions and Results

The following table showcases a brief summary of comparison results.

	KNIME	MS Power BI	Google Colab	Streamlit	Gradio	DataRobot AI Platform	PYTSBE	This Project
Desktop Version	+	+	+	+	+	+	+	+
Mobile Version	-	-	+	-	-	+	-	+
Does not require coding skills	+	+	-	-	-	+	-	+
Support for Python libraries	-	-	+	+	+	-	+	+
Cloud-based	-	±	+	-	-	+	-	+
Open Source / Freely Available	+	-	+	+	+	-	+	+
Supports Time Series Assessment Workloads	±	±	+	+	+	-	+	+
Supports Forecasting of Time Series	±	-	+	+	+	+	-	+

Table 2.1. Overview of features and aspects of the mentioned alternatives

Generally speaking, there is no proper solution for a freely available system on the market, which relieves the users from the need to write code for the purposes of performing time series analysis across different platforms, and this is precisely the aim of this project to remedy.

3. Requirements

3.1 Functional Requirements

- Authentication: the platform is equipped with standard algorithms for accomplishing user authentication via the means of login and password provision. Anyone is able to get registered within the system, gaining access to publicly posted content and the ability to create their own.
- Project creation and management: the platform permits its user for the creation and customization of Project objects within the system, which act as an encapsulating datatype for holding most of the relevant information regarding the data processing pipelines.
- Permissions and administration: the application manages and controls which users are authorized to view certain projects, datasets and similar. This is to ensure that the privately set works within the system remain only accessible to its authors. Furthermore, only authors are able to actually edit the data processing pipelines, so the system extends the defined access controls over to the API as well.
- Management of uploaded data: Uploading, storing and accessing datasets from within the system (through the pipeline editor in particular), as well as storing additional metadata for ease of navigation and management from within the system.
- Interactive data processing pipeline editor: mentioned above, this is part of the application which takes care of the construction of data processing pipelines, accomplishing said goal via visualizing available functions to the user, defining their parameters via forms, and executing them, all accomplished through an accessible graphical user interface rendered within one's device through a web browser.

3.2 Non-Functional Requirements

- Hardware: during the development of the project its environment was a fairly low-end HDD-based system with a fairly low-end CPU, yet the general functions of the site worked quite well, without any major performance hurdles or otherwise, largely granted by the great optimization work done on the side of utilized web servers and PostgreSQL.
- Cross-Platform availability: as per the goals of the project being a web application, it must ensure its availability across different devices and platforms. In part, this is set upon the general UI requirements, as for the project itself to be accessible among different devices, the UI must accommodate for them wherever possible.
- Feature set expansion: as per the design of the project's architecture, it should accommodate for the matter of addition of new functions for the purposes of being easily adaptable towards the implementation of new scenarios.

3.3 Conclusions and Results

The requirements described above present themselves as the general plan of the project's feature set during development. Combining different goals of the project's construction, covering different aspects of its usage and similar, a system capable of supporting different kinds of analytical and predictive workloads is being constructed, supporting data management, pipeline management, and more.

4 System Architecture

4.1 Technological stack

Considering the nature of this project being a Web Application, CSS, HTML & JavaScript are the tools that have been utilized for the purpose of its

development. As of today, these tools are already enough to construct extensive and adaptive designs adhering to currently supported browser rendering engines.

Certain aspects of these tools, however, remain inefficient, and so it was decided to employ the help of Svelte framework [5], which allows for splitting the Frontend application into Components. Each Component is responsible for handling certain kinds of logic within the Frontend application itself, as well as rendering certain parts of the user interface. Such act splits the application into more manageable code chunks, ensuring its maintainability and efficiency, as the complexity of the application grows.

For the task of enabling server-side functionality, upon which lay the tasks of data storage, user and data management, data processing, model training and utilization, Django framework has been selected [1], as it allows for rapid development of features required, while also ensuring that the safe practices are being followed, with the provision of safe and secure authentication patterns, CSRF token validation, XSS attack prevention and more. Furthermore, since it is based on Python programming language, it allows for extensive use of existing software solutions present in its repositories, primarily focused on data processing, modeling and its visualization.

For the matter of constructing the communication layer between the Frontend and the Backend in places where Django cannot handle required communication by itself, an additional tool under the name of Django REST framework [2] was employed, as it aids with the construction of REST-compliant APIs essential for proper interoperability required by the project at hand. It enables the functional parts of the Svelte-based editor application within the system, tightly integrating with the original Django framework, reducing complexity of development and increasing the maintainability of the project in the process.

For the purposes of storing and managing saved data, PostgreSQL [4] was chosen as the database engine for this project. The choice of this particular

solution was justified by its ease of integration into Django, with the latter supporting the object relational mappings to the submitted data, as well as high performance, storage efficiency and stability of its interface. Focusing on the general-purpose data storage, it is capable of serving all kinds of data required, including time series, images, additional metadata and more, all required for usage in this project.

Additionally, since it is a relational database, it allows for extensive construction of relations between submitted data into itself, which permits construction of extensive data structures suitable for complex workloads. In this project in particular, as it takes form of not just a simple application for data processing on the web, but as a platform, such functionality is essential to keep it operational and efficient.

4.2 Other Components Considered

This chapter is called for assessment of existing solutions, which were considered for the purposes of constructing the system at hand, yet were decided against for various reasons. The database engines considered in the following text were picked as displayed on DB-Engines Ranking [12], and as such have been additionally analyzed for the matter of achieving the set-out goals for the project.

4.2.1 Plain Django

Covering both parts of the backend and the frontend, Django could be utilized solely by itself to accomplish the final objective (coupled with PostgreSQL for data storage and regular CSS, HTML & JS for the frontend), it was ultimately decided to extend it with additional frameworks like Django REST and Svelte, for the matter that the editor portion of the project is far too complex to be implemented in simple JavaScript. Therefore, the decision was made to opt for a more complex software stack, yet the one that would take care of this issue.

4.2.2 SvelteKit

SvelteKit [13] – considering the usage of Svelte in the application, SvelteKit is generally considered as the primary backend choice for the stack, due to the benefits of their co-integration, simplifying API construction and reducing the general complexity of the code. However, considering that the primary concern of this project is the enabling of data science workflows, most of it is accomplished on the Python programming language with its own set of tools, which would not work with this stack since the backend portion of it works on JavaScript, and additional integration of the required feature set would involve creating another Python runtime on the backend, which can be circumvented by simply using Django.

4.2.3 Flask

Flask [14] is a Python based lightweight web framework that was additionally considered for the development of this project, however taking into account that it was built to be as simple and lightweight as possible it doesn't supply enough of the necessary features to make the project not only work and adhere to the safety standards of the modern day, but to also scale if the need is be.

While generally possible to accomplish this entire project via the means of Flask, Django provides a much more cohesive and robust toolset for the job, whilst taking care of quite a lot of aspects of work in the meantime, so it is therefore was not utilized in the development of the application in question.

4.2.4 NoSQL Databases (MongoDB, Apache Cassandra, …)

The use of non-relational databases like MongoDB, Apache Cassandra [15, 16], and similar was also considered with the development of this project, however it was decided against their use due to the nature of data handling in the project like this one, for that the underlying architecture of the database requires frequent referencing between different objects. Non-relational

databases do not supply the means to tie the datatypes together with relations, often instead offering simple dictionary-based approach for data management, which offers additional performance if the matter of the system's usage presents itself as repeated access of large volumes of low complexity data. Lastly, it poorly integrates into the current tech stack, and so it was opted to instead use PostgreSQL, with which comes better support within the selected Django framework, as well as enabling mapping of relational data.

4.2.5 InfluxDB

InfluxDB [17] is a popular database engine utilized for the purposes of storing and managing time series data in particular, as it is specifically optimized for the task, splitting the loaded datasets into separate points in time, accessible individually. With its benefits, it is often utilized for the purposes of process monitoring, metrics of application runtime and results, as well as performing real-time analytics.

For the matter of this project's development, it was decided against its usage, because while it is indeed optimized for the task of processing time series data, it lacks the supporting architecture for storing regular tables of data required for operating a fully-fledged website. Furthermore, its support in the current Backend of choice is lacking, in large part due to the issue described above. In order to take advantage of it, a manual integration with its own Python interface is required instead, which overcomplicates application architecture for little return in the scope of this project, since the system is meant to be public-facing into the internet or internal networks, therefore accessing external instances of InfluxDB is impractical and insecure, requiring such instances to be open to the public internet, posing a security threat to the data stored within.

It is possible for a regular relational database and InfluxDB to be present at the same time, however this comes at an increased cost in terms of performance of the system and databases, as well as major increase in the underlying complexity. This use case would only be optimal whenever the large quantities

of time series are already present within any particular InfluxDB instance, situated in a closed-off network, in junction with other services for which it performs the logging. Therefore, the usage of this database engine was opted against.

4.2.6 Apache Pinot

Apache Pinot [18] is another database engine optimized for storage and accessing large quantities of data with minimal latency. Unlike InfluxDB, however, it adheres to the Online Analytical processing (OLAP) approach, which minimizes time spent on querying for data, as such an architecture allows for much faster data lookups, as well as increased interoperability with other Apache products and systems.

Unfortunately, it succumbs to the same problems, as described with InfluxDB, being that it is unable to support the data structures required in this particular project. Additionally, its use and proper support in Django is similarly lacking, and, as it was described in InfluxDB, running several database engines for this particular project is inefficient, therefore it was decided to opt against its usage.

4.2.7 TimescaleDB

TimescaleDB [19] is a database engine, built upon and extending the feature set of PostgreSQL, the current database engine of choice within the project. This database engine provides additional data types catered towards time series analysis workloads, while also promising to deliver increased performance compared to that of PostgreSQL.

Considering this, it’s important to highlight, on the other hand, that the database engine is fairly new, the fact of which doesn’t inspire confidence in its stability, and is likely to bring up potential issues going down the road. As well as that, its support in Django isn’t as great in comparison to PostgreSQL, and coupled that its upside of handling deserialized time series data doesn’t propose

much of a benefit since in the system itself datasets are being handled in their serialized forms, it was ultimately decided to opt for the more tried and truer PostgreSQL engine.

4.3 Constructional Patterns

Since the project is intended to be a web service, available to the public or deployed within a local network, it adheres to the standard client-server model, in part described in sections above through implementation of the Backend, taking care of server-side logic, which includes processing functions definitions and the runtime, and the Frontend, taking care of the user interface served to the user, with their browser acting as a client, handling certain logic by itself and communicating with the server.

4.3.1 Site Layout and Feature Overview

The resulting website is split into different directories, each one of them serving different purpose within the system, as showcased in Fig. 4.1.

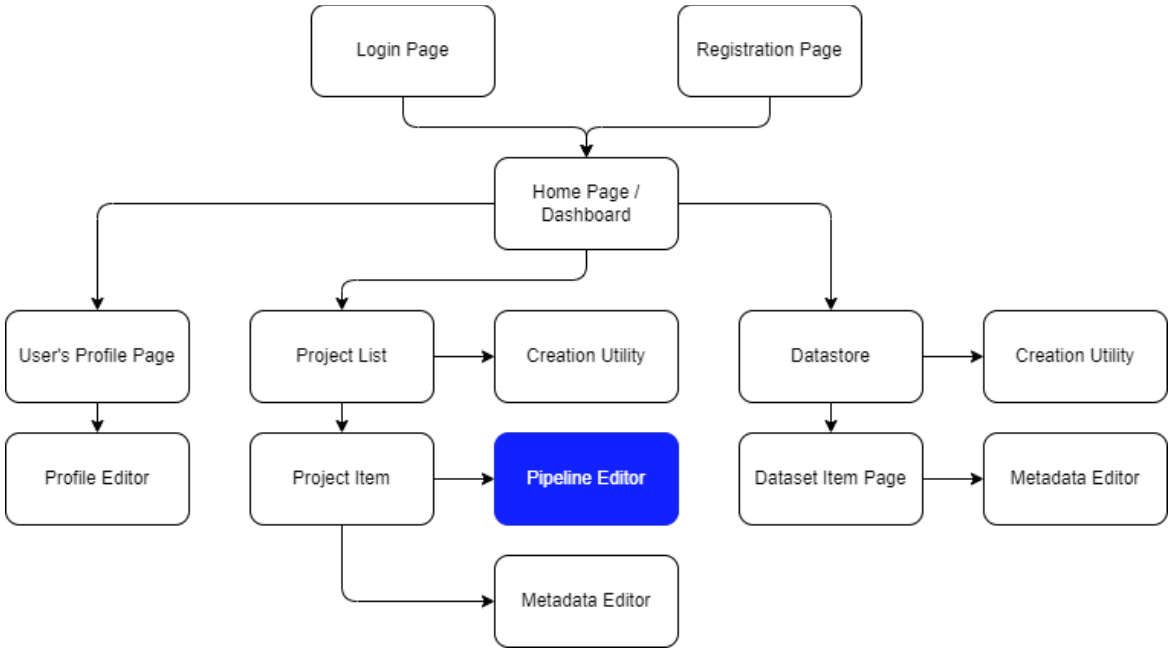

Fig. 4.1. The map of the site's features

Upon first access to the site the user is greeted with a login/register screen, through which they must either authenticate themselves via filling out a

presented form, or switch to the registration form, as showcased on Fig. 4.2, in which they can create a new account. This ensures that the users of the site have an account while using the service, which allows for tracking of their behavior around the site, enabling the features of access control to specific projects and similar. Users' passwords are stored in an encrypted form in the site's database in accordance to the modern safety standards on the web.

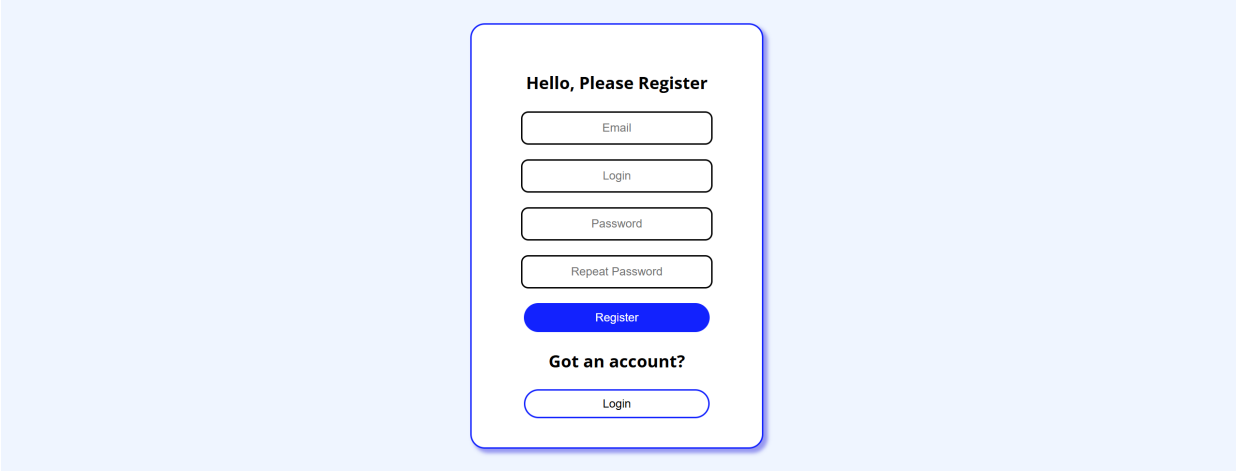

Fig. 4.2. Registration page

Proceeding further, after authenticating the user is greeted with a home page, presenting itself as a dashboard, split into three panels, as depicted on Fig. 4.3.

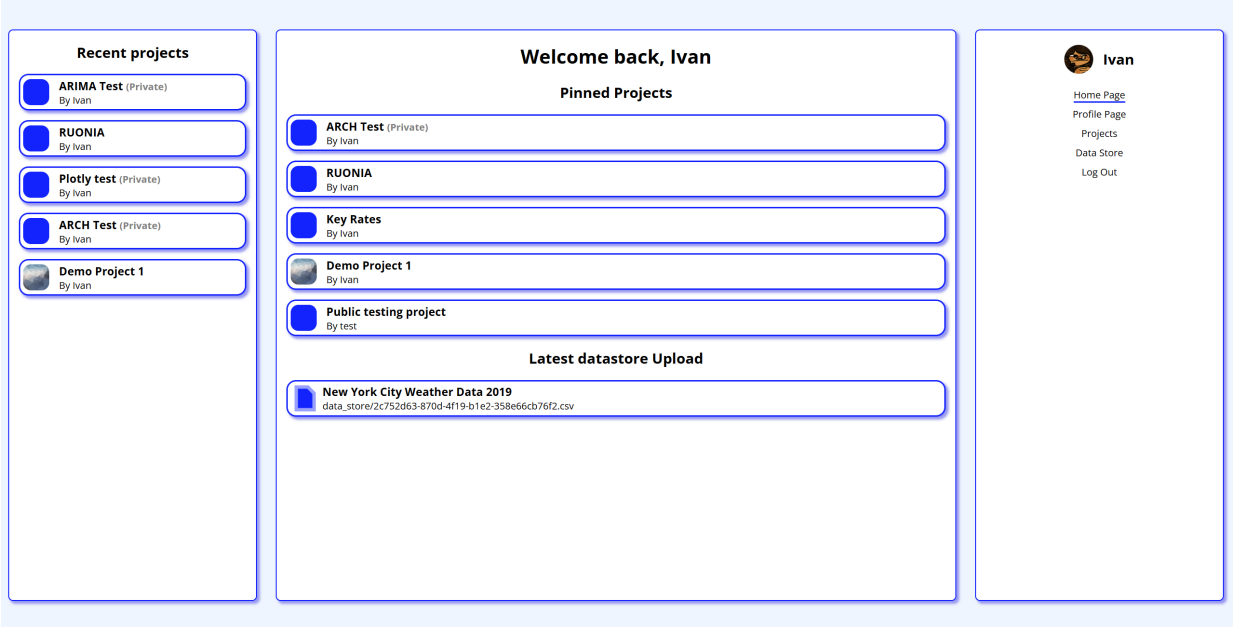

Fig. 4.3. Example of a user's home page

The dashboard is split into 3 different panels, which persists while the user is browsing the site for quick access and convenience. The left panel holds recently accessed projects, so that one could always resume their work. The right panel serves the function of a navigation bar, allowing the users to access different parts of the website.

The main panel is dynamic, serving different functions on each page. On the home page, it displays projects that the user might have pinned for quick access, as well as recently uploaded datasets for reference.

Proceeding along the links along the right panel, the user can access their profile page, on which they're able to see all projects with their authorship, as well as edit their appearance within the system, setting their profile picture, display name, as well as changing credentials like email, login and password if necessary. The profile page also contains a search bar, aiding in finding needed projects to work on.

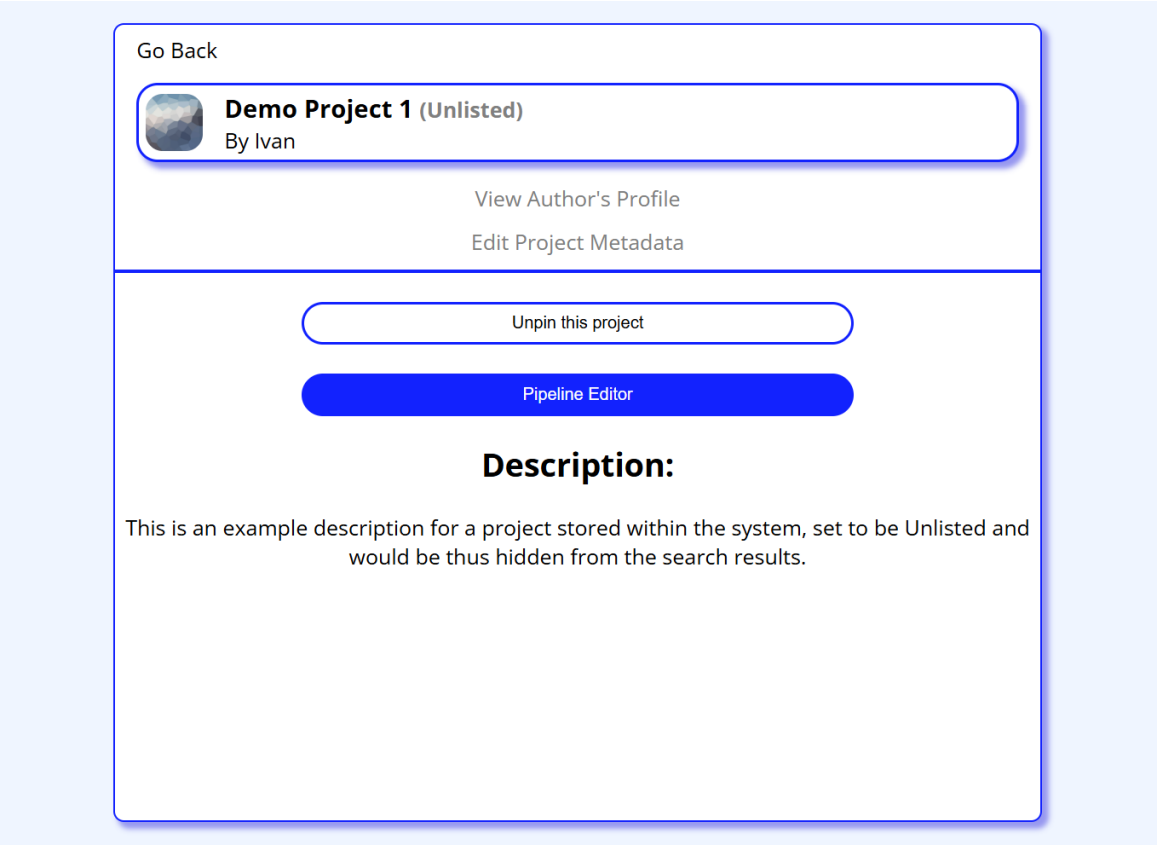

Fig. 4.4 Project Page inside the system.

The projects tab allows the user to browse projects freely available within the system for the purposes of easy access. Like with the profile page, it features a search bar, allowing users to find their projects of interest with relative ease.

Clicking on any of the project items present within the aforementioned menus will bring the user to the project page, as shown in Fig. 4.4, on which they are able to see its description, as well as access the pipeline editor for executing the data processing pipelines set up by the project authors. On this page, they are also able to pin the project to the dashboard if such a desire is present. If the currently logged in user is the author of the selected project, they are able to edit its metadata, changing its name, description and thumbnail. Furthermore, they are permitted to access the editor mode of the data processing pipelines related to the project object.

Going further, in the datastore section, the user is able to upload and manage their own datasets, with the aid of a search bar and a general list of all the datasets uploaded by the logged in user. Each one, once again, holds some metadata to aid with their management, namely a dedicated name and its description. These objects can be accessed from within the pipeline editor mode mentioned previously.

To start working in the system, the user should:

1. Either log into their account or make a new one within the system.
2. Create a new project.
3. Upload the dataset through the datastore section of the website.
4. Open the project's pipeline editor.
5. From within the pipeline editor, select desired functions, define their parameters to add them to the pipeline.
6. Run the pipeline, assuming that it is properly set up, and get the results. The results are stored within the project and available for later review, until the pipeline is executed again.

Lastly, the user is able to log out if the need is be, bringing them back into the login menu.

4.3.2 Security

Throughout the construction of the website, security has been considered in order to ensure safe and proper storage and handling of user generated data within the system.

On the matter of user authentication, or, to be more specific, the matter of password storage, they are, as mentioned before, stored in a database in an encrypted and hashed form. Every password submitted is processed with the Password-Based Key Derivation Function 2 algorithm and further hashed with the help of a SHA256 algorithm, ensuring their secure storage sufficient for the site's operation and user safety, as this algorithm chain is recommended by National Institute of Standards and Technology [20].

In addition, this procedure also establishes a CSRF token on the client side, required on every submission of user data and changing internal states, as is necessary to prevent Cross-Site Request Forgery attacks. A CSRF attack is a type of an attack whenever a malicious actor steals authorization cookies from a user and then subsequently proceeds to perform requests to the target website without the user themselves authorizing such actions, or even being aware of it. A CSRF token validates the origin of a request, and as such stops these attacks from occurring. This project utilizes this protection mechanism in all of its parts, which are capable of altering database states and similar.

In regards to user management within the system, there are 2 general roles set up for the purposes of controlling access to certain parts of the website: regular user and administrator. Regular users are only allowed access to the public facing portion of the website, on which the access to items within the system is limited by the set-up visibility settings, as well as project authorship

(those who create the projects in question retain access to the ability to edit their metadata and pipelines. Same goes for the uploaded datasets).

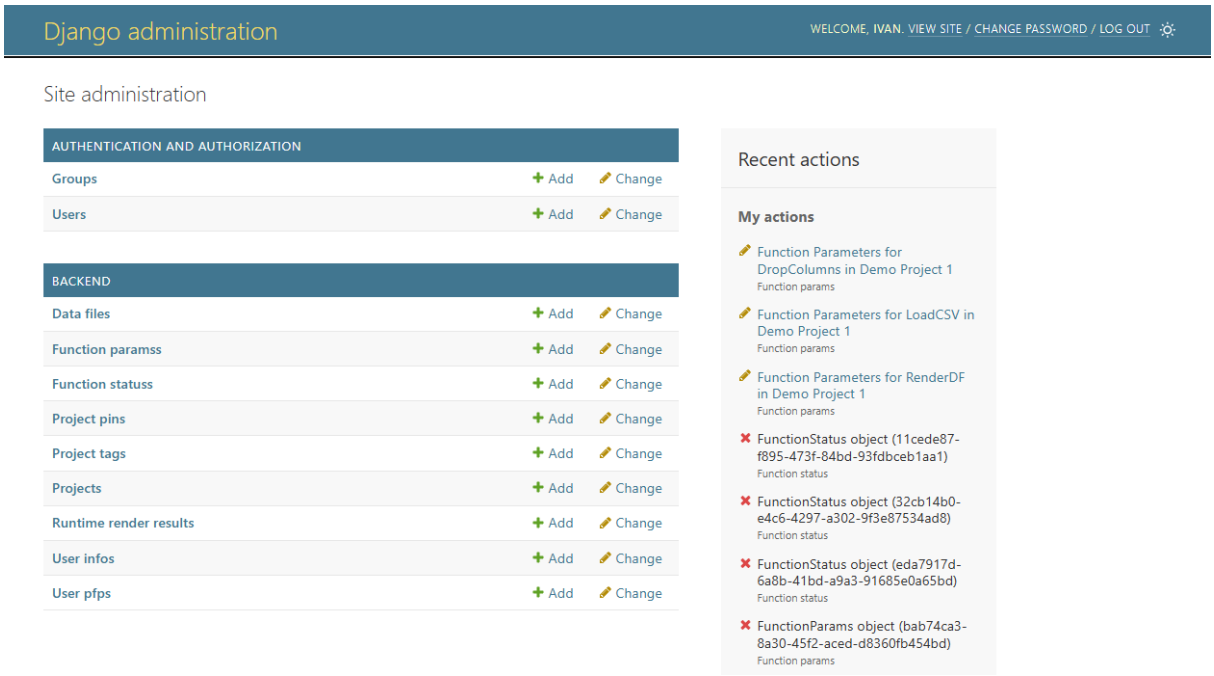

Fig. 4.5. Administration panel of the system.

This front-facing portion of the website remains generally the same for administrator users with one key difference, that being that they are permitted to access the administration panel of the site, showcased in Fig. 4.5, on which they are able to browse and edit objects stored in the database directly, such as user profiles, their project items, databases, and more. This role exists for the purposes of system administration. In addition, it can also be utilized to provide support for the site, helping with remedying potential issues that might arise during the work conducted by the users on the site.

By default, created projects are only available to browse and see to their authors. The visibility settings can be adjusted to share them with other users either via link access or setting it to public, in which case these projects will begin to appear in system's search. Projects marked as private are not accessible to anyone aside from system administrators and authors of said projects.

Whenever a project is set to be publicly shared within the system, other users are able to browse the pipeline established by the work of the author, view

all of the set-up functions present, their parameters and more. They are also able to see the last result of the pipeline execution within the same page, however they are unable to edit any of the functions set up, as is implemented to prevent any potential harm done to the work of other users.

Similar control system is implemented with regards to the datasets uploaded into the system. Unlike project items, however, uploaded dataset objects are visible and accessible only to those users who have uploaded them in the first place, for the purposes of safety and security of the data being handled. That said, if the users wish to share datasets between each other, they can set them up for proper display within the pipeline editor.

4.3.3 Data Processing Pipeline Editor

The editor part of the application, mentioned in 4.3.1 allows the project authors to set up data processing pipelines via a graphical interface. While on the editor page, as showcased on Fig. 4.6, the side panels change their functions: the left panel begins to display functions available within the system that can be executed; the right panel shows the order of the constructed data processing pipeline, and the main panel now displays the parameters of each function that the user chooses to select.

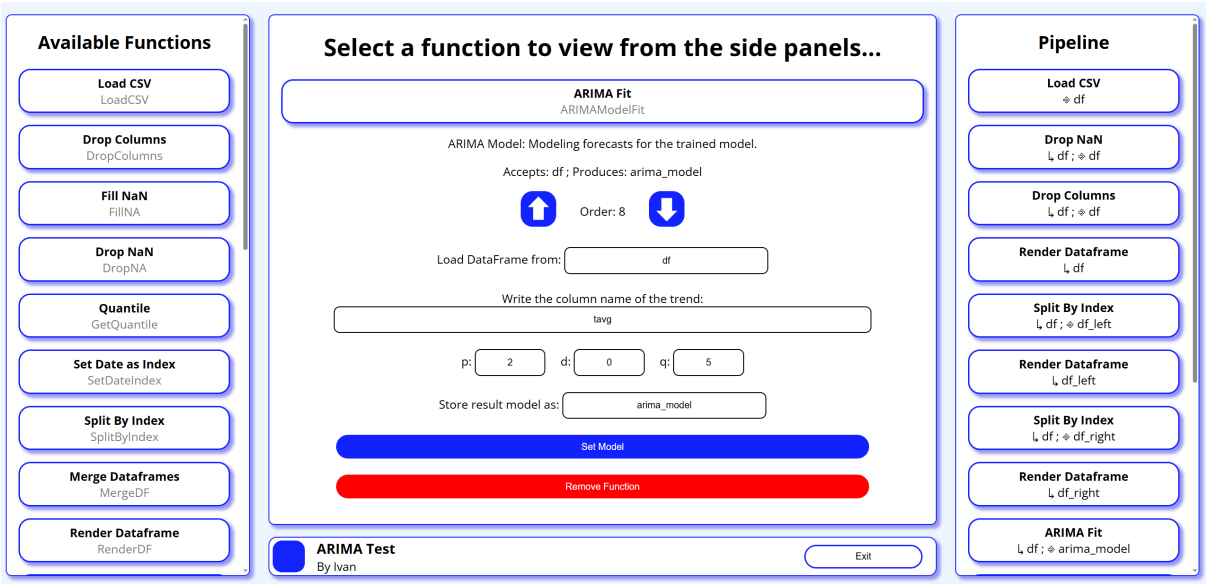

Fig. 4.6. Editor Page showcasing ARIMA Fit function form.

Every function supplies their own parameters to be filled out by the user, as well as the brief description of its functionality, and more technical information on the matter of what variables it accepts, what values does it produce, loading of datasets and rendering information onto the user's screen. It also features an ability to move the pipeline functions around if the need for that is present.

As the user finishes up their work on this matter, and the pipeline is validated so that there is both data present in its runtime, as well as there are output functions present, the user can now press a button on the right panel in order to invoke pipeline's execution. From this point on, the function UI locks up, and, upon the completion of the runtime on the server side, the user is presented with the result, defined by the pipeline functions and parameters. From there, they can study this result, get appropriate data for themselves, and similar.

In addition, if the author of a particular project has set its visibility setting to either public or unlisted and is sharing it with other users, said users are able to browse the pipeline set up in a special locked down mode of the editor. While in this mode, they are still able to see the pipeline itself, browse its set up parameters and view the results of the last runtime execution if they so wish.

Once the process is complete, they can reset it and either get back to work on the pipeline, or leave the editor. All of their work is stored within the system's database, accessible to the author at any time.

4.3.4 User Interface Design

The UI has been constructed with a unified theme reflected in the general layout, iconography and design language.

The aforementioned 3 panel structure of the application is designed to make a uniform framework of laying out items in the application, which stays consistent throughout the site. It also enables the increased adaptability of the website for mobile devices, as with small screen sizes the side panels disappear,

yet can still be invoked via the use of newly appearing buttons on the top right corners of the main panel. This is to ensure that the service remains accessible to people from different devices and platforms, minimizing design obstructions which make it hard to use in certain conditions.

Lastly, it aims to maintain a consistent theme for the purposes of general branding and recognition, needed to stay out in the market of existing solutions.

4.4 Data Processing Architecture Overview

This section of the chapter is called for the matter of elaborating upon the implementation of the data processing pipeline editor, mentioned in 4.3.1 and further elaborated upon in 4.3.2.

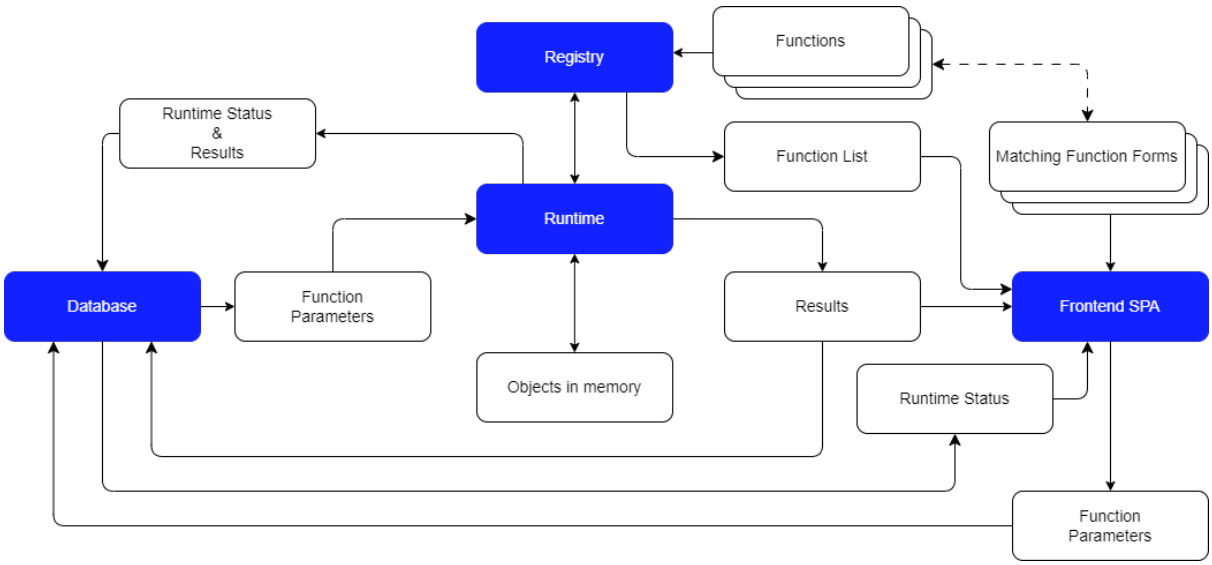

Fig. 4.7. Representation of the data processing architecture within the application

The data processor portion of the application is split into several components, similar to the rest of the website. Namely, the things directly responsible for its operation are the database, the function registry, the runtime, and, lastly, the Frontend SPA.

The database, the function registry, and the runtime are both situated on the server side, with the latter two being implemented within the Backend framework of the application, that being Django. The database interactions are

being handled through Django on the server-side, thus all of the interactions with it are being handled through another layer of logic. The general interactions between the systems in place are depicted on Fig. 4.7.

Covered before, the database acts as a storage for many things related to the pipeline processor, specifically taking care of storing user defined pipelines for their processing in the runtime, as well as acting as a log, storing results of pipeline execution, together with the intermediate states of the functions. All of this gets queried by the Frontend SPA to display the status of the work to the user. The ER diagram showcasing the structure of the website's database is showcased in Fig. 4.8.

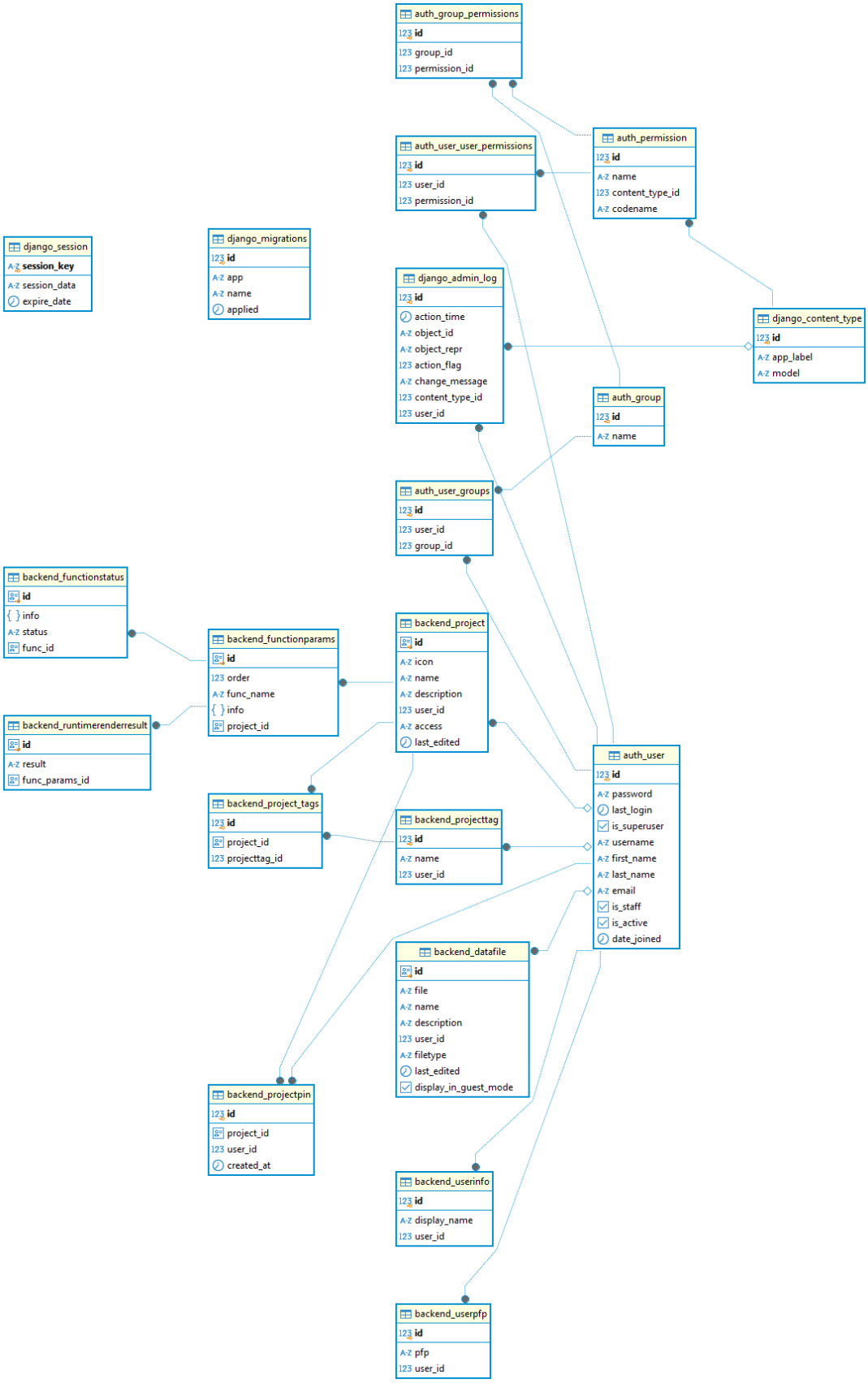

Fig 4.8. ER diagram of the website's database

The runtime portion of the application is a special function situated within the logic of the Backend, which takes care of the actual execution of the pipelines defined by the user, in accordance to the supplied parameters. In order to accomplish said task, it iterates over the defined pipeline, getting the function parameters from the database, previously deposited there via said user's actions in the Frontend SPA. After retrieving the pipeline order and function parameters, the runtime creates an object in the memory in which it holds all of the intermediate values created and consumed by the functions themselves, and thus begins the execution process. On each step, it logs a special object into the database which contains information on the function execution status (whether it has been executed or not, and its success status, errors, additional information whenever necessary). As the runtime continues to operate, the Frontend SPA is actively querying the backend for updates on the matter of runtime execution, which it receives based on the statuses logged in the database. If any errors arise during the runtime, caused by improper assembly of the pipeline by the user, the execution halts and the Frontend SPA retrieves the error to display on the client in order to help with its remedying. If a function produces a result to be displayed to the Frontend SPA, the runtime saves it separately within the database. Lastly, when the execution of the pipeline is finished, the Runtime notifies the Frontend SPA of its successful completion, so that the results can be freely retrieved for their display and studying.

In order to actually execute the functions stated within the pipelines, however, the Runtime requires additional help from the registry object, situated within the Backend functions. It acts as a mapper to the actual functions that can be executed, as well as a way to access their metadata, holding information on the matter of accepted and produced variables, function types, descriptions, names, etc. Utilizing Python's features on the matter of referencing functions as objects, it acts as an integral part of the data processing architecture of the system, as it allows for execution of defined functions in an arbitrary way, as required by the nature of the work.

On the part of the Frontend, there lies a special SPA, which takes care of the client logic within the defined data processing architecture. Its use lies within providing the user with the means of actually constructing the data processing pipelines to be handled by the previously described parts of the architecture, as well as performing additional logic that can be ran on the client side (for example validating that the variables that certain functions intend on processing are actually being created within a pipeline.

In order to enable construction of the data processing pipelines, there needs to be a graphical representation of functions present on the web page that the users access. As such, the Frontend SPA is responsible for portraying all of the available functions within the system for the users to work with. In particular, since each function does its own unique things, the application supplies special forms for every one of said functions. The forms load into the application as components, with a standard model of their operation, whilst containing all the Frontend logic within themselves so to not interfere with the rest of the application. As the user completes their work with any particular function, it can be set to be accepted within the pipeline, in which case the aforementioned forms get sent back to the Backend to be handled based on their types and supplied data, later to be stored into the database for the runtime to work with. With this, the pipeline gets updated and validated, for the user to either continue working on, adding new or editing existing functions, or to execute, putting the rest of the system in action.

Described and implemented as such, this structure allows for flexible execution of the data processing pipelines within the system's framework, as well as ensuring the capabilities for its feature expansion, ensuring that the only boundary for successful implementation of additional functions remains to be the capabilities of the Python language and its libraries. While maintaining that, it also puts an effort into keeping things simple to use and understand for its

users, allowing for the flexibility of its usage, as well as the ease of it at the same time.

4.5 Conclusions and Results

By the end of the work, the system described in the chapter has been successfully obtained, utilizing the tooling mentioned in the technological stack, in order to bring forth the goals set out by the system requirements laid out in chapter 3, striving towards the goal of creating a system for easy construction and management of data processing pipelines with the primary goal set forth upon short-term forecasting of time series data.

5. Models

For the purpose of demonstrating the capabilities of this project, the models suggested by my project partner, Ilya Suchkov, are utilized, selected by him as a part of a professional assignment. The following chapters contain the brief descriptions of the models, as well as general considerations with their implementation.

5.1 Models Utilized

For the matter of showcasing the system's capabilities, various regressive models have been supplied as part of the pipeline editing functionality. Such models have been selected due to their general popularity within the field of time series analysis and forecasting workflows, as well as their relatively low demand for computational resources. Such models include the general Autoregressive model, ARIMA model, the ARMA model, the ARCH process and its derivatives (GARCH, ARCH, EGARCH, etc). In addition to this, the model work performed by my project partner as part of his portion of the project has also been supplied.

As it was described before, the system at hand is built with further expansion in mind, therefore additional models for various purposes can be brought onboard for any particular task if the need for it remains.

5.2 Performance Comparison

One of the possible use cases considered for the system was not only the act of performing simple forecasting and predictive analysis of time series data stored and processed within the system, but also the comparison of existing models on the same datasets for proper assessment of various metrics, which can be useful for the purposes of selection of certain models for particular workflows.

As described in the paper "TSPP: A Unified Benchmarking Tool for Time-series Forecasting" [21], the task of properly benchmarking various models is a complicated endeavor for the matter of vast difference of types of how different models perform. As such, mentioned in the paper, a collective of metrics utilized is important for the matter of proper assessment of various model performances. The general conclusion about the assessment of models, then, is the showcase of various kinds of metrics, available for the testing of the models.

For the purposes of demonstrating the assessment performances of different models, and considering the fact of metric deviations described in the paper above, it has been decided to include most of the metrics capable of being performed on the selected models. The list of such metrics includes: R^2, Log-Likelihood, AIC, BIC, among others. Aligning them on different models situated within the system, performance assessments can be made. Furthermore, the results of their calculation are also stored within the system for later analysis if the need for such purpose is be.

5.3 Conclusions and Results

Overall, a set of models for showcasing the capabilities of the system for the matter of performing various types of analysis, as well as accomplishing the

predictive workloads of the time series forecasting are the goals that have been achieved. Together with this, additional facilities for performing comparisons between established models have also been obtained, allowing users to properly assess various results obtained during their interactions and work in the system.

6. Conclusion

During the development cycle of the project, a system, the feature set of which allows for performing of short-term forecasting of time series, utilizing various functions and models at hand, has been achieved. The system is adapted towards operating across various platforms and can freely be utilized by users without prior the knowledge of coding.

The ability to expand the system's features with additional models allows for its utilization in various kinds of scenarios. As one of the scenarios for the usage of the system, implemented regressive models can be utilized for the matter of assessing the changes in trends in the short term, while for the more complex scenarios where precision is paramount, models of the transformer nature and similar can be supplied and utilized within the system's framework.

The set-out goals on the matter of achieving the technical requirements for the system have been successfully achieved as well. The system features the required tooling for proper project creation, management, access control and more within itself, supporting the proper construction of data processing and time series forecasting pipelines. Lastly, to accommodate for the matter of the security concerns, the proper steps have been taken to ensure the safety of the data stored within the system itself.

The source code of the project is available at the following link: https://github.com/Miv2nir/diploma2425.

6.1 Further Development

For the matter of further development and improvement of the system at hand, the following directions can be undertaken:

- Creation of additional roles within the system to be used by the customer service (helpdesk team) instead of administrator roles.
- Development of additional APIs for the matter of exporting data into other visualization systems.
- Automating analysis workloads through the invocation of user-defined data processing pipelines through additional API for the purposes of automating analytical workloads.

7. List of Sources

1. Django Software Foundation. Django Framework [Online Resource]. URL: https://www.djangoproject.com/ (Date of Visit: 07.03.2025).
2. Encode OSS Ltd. Django REST Framework [Online Resource]. URL: https://www.django-rest-framework.org/ (Date of Visit: 15.04.2025)
3. Python Software Foundation. Python Programming Language [Online Resource]. URL: https://www.python.org/ (Date of Visit: 07.03.2025).
4. PostgreSQL Global Development Group. PostgreSQL [Online Resource]. URL: https://www.postgresql.org/ (Date of Visit: 08.03.2025).
5. Rich Harris. Svelte [Online Resource]. URL: https://svelte.dev/ (Date of Visit: 16.04.2025).
6. KNIME. KNIME Analytics Platform [Online Resource]. URL: https://www.knime.com/ (Date of Visit: 25.01.2025)
7. Google. Google Colaboratory [Online Resource]. URL: https://research.google.com/colaboratory/faq.html (Date of Visit: 13.04.2025)

8. Snowflake Inc. Streamlit [Online Resource]. URL: https://streamlit.io/ (Date of Visit: 18.04.2025)
9. Hugging Face. Gradio [Online Resource]. URL: https://www.gradio.app/ (Date of Visit: 30.04.2025)
10. Microsoft. Power BI Overview [Online Resource]. URL: https://learn.microsoft.com/en-us/power-bi/fundamentals/power-bi-overview (Date of Visit: 04.04.2025)
11. DataRobot Inc. DataRobot AI Platform [Online Resource]. URL: https://www.datarobot.com/product/ai-platform/ (Date of Visit: 16.05.2025)
12. Red Gate Software Ltd. DB-Engines: Ranking of Time Series DBMS [Online Resource]. URL: https://db-engines.com/en/ranking/time+series+dbms (Date of Visit: 28.04.2025)
13. Rich Harris. SvelteKit Documentation [Online Resource]. URL: https://svelte.dev/docs/kit/introduction (Date of Visit: 07.03.2025)
14. Armin Ronacher. Flask Documentation [Online Resource]. URL: https://flask.palletsprojects.com/en/stable/ (Date of Visit: 07.03.2025)
15. MongoDB Inc. MongoDB [Online Resource]. URL: https://www.mongodb.com/ (Date of Visit: 08.03.2025)
16. Apache Software Foundation. Apache Cassandra [Online Resource]. URL: https://cassandra.apache.org/_/index.html (Date of Visit: 08.03.2025)
17. InfluxData. InfluxDB Time Series Data Platform [Online Resource]. URL: https://www.influxdata.com/ (Date of Visit: 28.04.2025)
18. Apache Software Foundation. Apache Pinot™ [Online Resource]. URL: https://pinot.apache.org/ (Date of Visit: 15.05.2025)
19. Timescale Inc. Timescale: PostgreSQL++ for time series and events. [Online Resource]. URL: https://www.timescale.com/ (Date of Visit: 28.04.2025)

20. Turan M., Barker E., Burr W., Chen L. Recommendation for Password-Based Key Derivation. Part 1: Storage Applications. // NIST. Special Publication 800-132. URL: https://csrc.nist.gov/pubs/sp/800/132/final (Date of Visit: 18.04.2025)
21. Bączek J., Zhylko D., Titericz G. TSPP: A Unified Benchmarking Tool for Time-series Forecasting // Cornell University, arXiv Platform. URL: https://arxiv.org/abs/2312.17100 (Date of Visit: 16.05.2025).
22. Natural Systems Simulation Lab. Pytsbe: Python time series benchmark. [Online Resource]. URL: https://github.com/ITMO-NSS-team/pytsbe (Date of Visit: 16.05.2025).